TypeScript Notes

What is TypeScript?

TypeScript is a programming language created by Microsoft that is a superset* of Javascript.

> ⭐ Superset = every valid Javascript program is already valid TypeScript i.e TypeScript is build on top of JavaScript

Key idea: You write TypeScript → It gets converted/compiled into plain JavaScript → The browser only runs JavaScript , never TypeScript.

Why JavaScript alone was not enough?

JavaScript created in 1995 was used for → small scripts , simple web interactions , few files , simple developer.

But today , JavaScript is used for → Large web apps , backend servers , mobile apps , teams of 100+ devs.

Problem with Plain JavaScript :

```
function add(a, b) {
  return a + b;
}

add(5, "10"); //returns "510" (BUG)
```

Problems :

- Bugs appear at runtime (too late)
- You only discover errors after clicking a button , after deploying a production , after users complaint.
- Hard to maintain large codebases , you don't know what a function expects , remaining breaks things silently , IDE autocomplete is weak.

Why TypeScript was created?

Microsoft created TypeScript (2012) to solve :

- Catch errors before running the code.
- Make JavaScript scalable for large applications.
- Provide string tooling (autocomplete , refracting).
- Keep JavaScript compatibility.

TypeScript doesn't replace JavaScript , it improves JavaScript.

How TypeScript solves JavaScript problems :

1. Type checking (core feature) : provides consistency

```
function add(a: number, b: number): number {
  return a + b;
}

add(5, "10"); // Error before running

function greet(name: string): string {
return `Hello, ${name}`;

console.log(greet("Harry"); // Hello, Harry
```

2. Compile-time safety :

   TypeScript checks code at compile time , not runtime.

How TypeScript works (behind the scenes)

▼ TypeScript Code

Code written in TypeScript

▼ Lexer

Tokenizes the code → detects basic errors in syntax.

▼ Parser

Generates an AST (Abstract syntax tree) of the entire code or a type of visualization of the complete code

▼ Binder

Takes AST and create Symbol Tables , Parent Pointer to navigate in tree and Flow Node (if else of nodes).

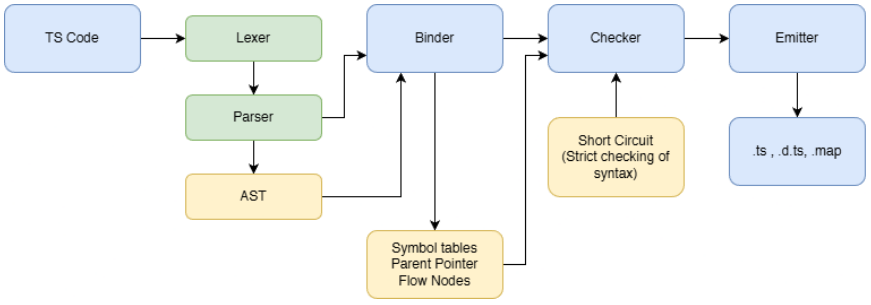
▼ Checker

Goes through twice in the code, and check every symbol to assure datatypes are correctly mentioned and assigned. Creates a strict syntax check.

▼ Emitter/Generator

Generates files and do stripping (removes the extra stuff that TypeScript added). Ex: node.js does this directly.

▼ .js , .d.ts , .map



Installation Process :

Can be done globally or project wise but the later is more recommended as we may require different versions of TypeScript in different projects.

```
npm init -y
npm install -D typescript (temporary)
npx tsc –init (generates tsconfig file , as the node module
s provide us with tsc → typescript compiler)
npx tsc ( to compile the .ts code in ./dist directory where
index.js is generated )
node dist/index.js ( to run the index.js file using node )
```

Type Inference in TypeScript :

TypeScript automatically figures out the type of a variable, function return, or expression without you explicitly writing the type. This is one of the most important features of TypeScript.

```
let count = 10
count = 20 // OK
count = "ten" // Error

// in JavaScript
let value = 10
value = "hello" //allowed

// in TypeScript
let value = 10
value = "hello" //compile-time error

// in TypeScript
let numbers = [1,2,3]
numbers.push(4) // OK
numbers.push("five") //WRONG
```

Type Annotations in TypeScript :

Type Annotations are explicit type labels you write in your code to tell TypeScript exactly what type should be.

```
let age: number = 20

//arrays
let scores: number[] = [1,2,3]
// OR
let score: Array<number> = [1,2,3]

//objects
let user: {
name: string
age: number
} = {
name: "Harry",
```

```
age: 22
}
```

Union and Any in TypeScript :

'Union' can be used to pre-define the values of a variable

```
// union
let subs: number | string = '1M'

let apiRequestStatus: 'pending' | 'success' | 'error' = 'pe
nding'

apiRequestStatus = 'done' //Error
```

'any' is used when you intentionally don't want to care about the type.

```
const orders = ["12", "21", "44", "26"];
let currentOrder; //type - any
let currentOrder: string; //ideal case
let currentOrder: string | undefined; //considering all use
cases

for (let order of orders) {
  if (order === "26") {
    currentOrder = order;
    break;
  }
  currentOrder = "11";
}

console.log(currentOrder);
```

Type Narrowing in TypeScript :

Type Narrowing is the process where TypeScript reduces a broad type into a more specific type after you perform checks in your code.

Problem :

```
let input: string | number;
input.toUpperCase() // Error - input can be number too
```

Type Narrowing methods :

- **'typeof'** Narrowing (most common) :

```
function normalizeInput(value: string | numeber) {
  if (typeof value === "string") {
    return value.trim().toLowerCase();
  }
  return value.toFixed(2);
}
```

- **'Truthiness'** Narrowing :

```
function showUserName(name?: string) {
  if (name) {
    return name.toUpperCase();
  }
  return "Annonymous";
}
```

- **'Equality'** Narrowing :

```
function formatPrice(price: number | null) {
  if (price === null) {
    return "Price not available";
  }
  return price.toFixed(2);
}
```

- **'in'** Operator Narrowing : using custom type

```
type SuccessResponse = {
  data: string,
};
```

```ts
type ErrorResponse = {
  error: string,
};

function handleResponse(res: SuccessResponse | ErrorResponse) {
  if ("data" in res) {
    console.log(res.data);
  } else {
    console.error(res.error);
  }
}

//here "data" in res is used as a check because TypeScript only knows res is either SuccessResponse or ErrorResponse , there is no guarantee that data exists.
```

- **‘ instanceof ’** Narrowing :

```ts
function handleError(err: unknown) {
  if (err instanceof Error) {
    console.log(err.message);
  } else {
    console.log("Unknown error");
  }
}

//err instanceof Error means -> Is err an object created from the built-in Error class (or a subclass like TypeError, ReferenceError, etc.)?
```

- Literal Type Narrowing :

```ts
type Status = "loading" | "success" | "error";

function render(status: Status) {
  if (status === "loading") {
    return "Spinner...";
```

```
  }
  if (status === "success") {
    return "Data Loaded";
  }
  return "Something went wrong";
}
```

- Discriminated Unions (VERY IMPORTANT) :

```
type apiStata =
  | { status: "loading" }
  | { status: "success", data: string }
  | { status: "error", message: string };

function renderState(state: apiState) {
  if (state.status === "success") {
    console.log(state.data);
  }
  if (state.status === "error") {
    console.log(state.message);
  }
}

// ❌ weaker design
type ApiState = {
  status: "loading" | "success" | "error"
  data?: string
  message?: string
}
```

Forceful type assertion : we forcefully annotate the variables

```
let response:any = '32';
let numericLength: number = (response as string).length;

//here response was not specified to be a string, so to use
string methods on it we need to specify it as string
```

‘readonly’ property

This property can be read, but cannot be reassigned after creation. It enforces immutability at compile time

```
//creation
type User = {
readonly id: number //once set it cannot be changed
name: string
}

//usage
const user: User = {
id: 1,
name: "Harry"
}

user.name = "Harry khurmi" // allowed
user.id = 2 // Error
```

Interface in TypeScript :

An interface defines the shape of an object. It tells TypeScript what properties and methods must exist, but not how they are implemented. It guarantees structure correctness at compile time.

Interface is like a contract , you cannot miss/add anything from it.

```
//creation
interface User{
id: number
name: string
isActive: boolean
}

//OR

interface User{
id: number;
name: string;
```

```
isActive: boolean;
}

//usage
cost user: User = {
id: 1,
name: "Harry",
isActive: true
}

// can also define methods inside it
interface Machine {
start(): void
stop(): void
}

const m1: Machine = {
start(){console.log("START"},
stop(){console.log("STOP"}
```

In TypeScript interfaces, properties are separated by semicolons (;) or newlines, not commas.

Interfaces are compile-time only. After compilation :

- Interface doesn't exists.
- No JavaScript code is generated
- Used only by the TypeScript compiler
- Therefore, " user instanceof User " → Impossible

Where Interfaces are used in Real Projects :

1. Typing API responses

```
interface ApiUser{
id: number
```

```
email: string
}
```

2. Function parameters

```
interface LoginData {
email: string
password: string
username?: string  //optional properties
}

function login(data: LoginData){
//safe access
}
```

3. Classes (implements)

```
interface Logger {
  log(message: string): void;
}

class ConsoleLogger implements Logger {
  log(message: strong) {
    console.log(message);
  }
}
```

4. Interface extension

```
interface Person {
  name: string;
}

interface Employee extends Person {
  employeeId: number;
}
```

5. Interface Declaration Merging

```ts
interface User {
  name: string
}

interface User {
  age: number
}

//TypeScript merges them into:
interface User {
  name: string
  age: number
}
```

Partial & Required : core utility types

1. **Partial<T>** : Make all properties of T optional. It makes properties optional but doesn't make them undefined safe at runtime

```ts
interface User {
id: number
name: string
email: string
}

type PartialUser = Partial<User>
// is equivalent to
type PartialUser = {
id?: number
name?: string
email?: string
}

//Real example

function updateUser(id:number, data: Partial<User>) {
// user can update just 1 field
}
```

```
// usage:
updateUser(1, {name: "New Name"})
updateUser(1, {email: "abc@gmail.com"})
```

2. Required<T> : Make all properties of T required (removes ?). It guarantees no missing config and no runtime crashes

```
interface Config {
apiUrl?: string
timeout?: number
}

type FullConfig = Required<Config>
// is equivalent to
type FullConfig = {
  apiUrl: string
  timeout: number
}

//Real example
function startApp(config: Required<Config>) {
//safe: config.apiUrl always exists
}
```

3. Pick<T> : Make selected properties of T required.

```
interface User {
id: number
name: string
age: number
address: string
}

type UserInfo = Pick<User , "id" | "name">;

function getUser(userData: UserInfo){
```

```
//only requires id and name of the user
}
```

Difference in *type* and *interface :*

Interface is for defining and extending object shapes (contracts) , whereas type is for defining any kind of type, including unions and compositions.

What they have in common :

```
interface User {
  name: string
  age: number
}

type User = {
  name: string
  age: number
}
```

Where *interface* is better :

1. **Declaration merging:**

```
interface User {
  name: string
}

interface User {
  age: number
}

//this becomes

interface User {
  name: string
  age: number
}
```

2. Clean extension with *extends :*

```ts
interface Person {
  name: string
}

interface Employee extends Person {
  employeeId: number
}
```

Where *type* is strictly more powerful :

1. Union types (impossible with interfaces) :

```ts
type Status = "loading" | "success" | "error"
```

2. Intersections (&) :

```ts
type A = { a: number }
type B = { b: string }

type C = A & B
```

3. Primitives, tuples, functions :

```ts
type ID = number | string
type Point = [number, number]
type Callback = (msg: string) => void
```

How to use them together?

```ts
interface UserID {
  id: number
}

type UserWithRole = UserID & {
  role: "admin" | "user"
}
```

Runtime difference?

- Both are erased at compile time
- Zero runtime cost
- Used only by the TypeScript compiler

Tuples & Enums in TypeScript :

1. Tuples :

```
//let User: [string, number, boolean?];
let User: [name: string, age: number, married: boolea
n?];

User = ["Harry", 23]; //correct in order
//OR
User = ["Harry", 23, false]; //correct in order

User = [23, "Harry"]; //Error

// readonly Tuples
const location: readonly [number, number] = [23.44, 44.2
1]
```

2. enum : comes from the word “enumeration” meaning a fixed list of named values. An **enum** is a way to define a **set of named constants**.

   Def : An enum (enumeration) is a TypeScript feature used to define a fixed set of named constants, making code more readable, safer, and less error-prone.

   💡 enums do exist at runtime, unlike interfaces.

```
enum Role {
  Admin,
  User,
  Guest
}
```

```
function checkRole(role: Role) {
  if (role === Role.Admin) { ... }
}
```

- Numeric Enums : Default behavior

```
enum Status {
  Loading,
  Success,
  Error
}
//Under the hood, this becomes:
{
  0: "Loading",
  1: "Success",
  2: "Error",
  Loading: 0,
  Success: 1,
  Error: 2
}
// so
Status.Loading === 0
Status[0] === "Loading"
//This reverse mapping is unique to numeric enums.
```

- String Enums : More common today

```
enum Status {
  Loading = "loading",
  Success = "success",
  Error = "error"
}

Status.Loading === "loading"
// NO REVERSE MAPPING
```

- *const enum (*Advanced)

```ts
const enum Direction {
  Up,
  Down
}
```

- Union vs Enums :

use enum when	use union when
You need a runtime object	Only type safety is needed
You need reverse mapping	React props / state
Interfacing with legacy TypeScript/JS code	API status strings

> 💡 Most modern TypeScript code prefers union types over enums.

Generic type :

```ts
function wrapInArray<T>(item: T): T[] {
  return [item];
}
// T is a generic datatype

wrapInArray("masala");
wrapInArray(42);
wrapInArray({ key: "value" });
```

```ts
function pair<A, B>(a: A, b: B): [A, B] {
  return [a, b];
}
```

Generic interfaces :

```ts
interface Box<T> {
  content: T;
}
```

```
const numberBox: Box<number> = { content: 10 };
const stringBox: Box<string> = { content: "stringBox" };
```

OOPS in TypeScript

Classes :

```
class User {
  name: string;
  id: number;

  constructor(name: string, id: number) {
    this.name = name;
    this.id = id;
    console.log(this)// points to whoever created the new o
bject i.e User1 in this case
  }
}

const User1 = new User("Harry", 001);
```

Encapsulation : hiding internal details and exposing only what's needed.

```
class BankAccount {
  private balance: number

  constructor(balance: number) {
    this.balance = balance
  }

  deposit(amount: number) {
    this.balance += amount
  }

  getBalance() {
    return this.balance
  }
}
```

```
const acc = new BankAccount(100)
acc.balance // Error
```

Inheritance : Lets one class reuse and extend another

```
class Person {
  constructor(public name: string) {}
  //Parameter properties - is short for defining the variab
le and constructor separately
  //OR
  public name: string
  constructor(name: string){
  this.name = name
  }

  speak() {
    console.log("Speaking")
  }
}

class Employee extends Person {
  constructor(name: string, public employeeId: number) {
    super(name)
    this.name = name;
    this.employeeId = employeeId;
  }

  work() {
    console.log("Working")
  }
}

// here super refers to the parent(base) class i.e Person.
It means, call the constructor of the parent class and pass
name to it.
// why super is required?
```

```
//Because in JavaScript a subclass must call super() before
using this as super() sets up this
```

```
//Access modifiers in constructor parameters
class User {
  constructor(
    public name: string,
    private password: string,
    readonly id: number
  ) {}
}
```

super(name) calls the parent class constructor to initialize inherited properties.

public name in the constructor is a TypeScript shortcut that creates and assigns a class property automatically.

Polymorphism : Different objects respond differently to same method call

```
class Shape {
  area(): number {
    return 0
  }
}

class Rectangle extends Shape {
  constructor(private w: number, private h: number) {
    super()
  }

  area() {
    return this.w * this.h
  }
}
```

```
class Circle extends Shape {
  constructor(private r: number) {
    super()
  }

  area() {
    return Math.PI * this.r * this.r
  }
}

const shapes: Shape[] = [
  new Rectangle(10, 5),
  new Circle(3)
]

shapes.forEach(s => console.log(s.area()))
```

Abstraction : via interfaces and Abstract classes

```
interface Logger {
  log(message: string): void
}

class ConsoleLogger implements Logger {
  log(message: string) {
    console.log(message)
  }
}
```

'implements' vs 'extends' :

extends	implements
inherit from class	follow interface contract
class A extends B {}	class C implements D {}

TypeScript in React :

Typing functional component : Props typing

```tsx
type UserProps = {
  name: string
  age: number
}

function UserCard({ name, age }: UserProps) {
  return (
    <div>
      <p>{name}</p>
      <p>{age}</p>
    </div>
  )
}

//usage
<UserCard name="Harry" age={22} />
```

children prop :

```tsx
type CardProps = {
  children: React.ReactNode,
};

function Card({ children }: CardProps) {
  return <div>{children}</div>;
}

//usage
<Card>
    <h1>Hello</h1>
</Card>
```

useState with TypeScript :

```tsx
const [count, setCount] = useState(0) //TypeScript infers as number

//explicit union state
```

```
const [status, setStatus] = useState<"idle" | "loading" |
"success" | "error">("idle")
```

```
//Object State
type User = {
id: number
name: string
}
const [user, setUser] = useState<User | null>(null)

//acccess safely
if(user) {
user.name
}
```

Event Handling :

```
//Button click
function handleClick(e: React.MouseEvent<HTMLButtonElement
>) {
  console.log(e.currentTarget);
}

//Input Change
function handleChange(e: React.ChangeEvent<HTMLInputElement
>) {
  setValue(e.target.value);
}
```

Forms :

```
type FormData = {
  email: string
  password: string
}

const [form, setForm] = useState<FormData>({
  email: "",
```

```
  password: ""
})
```